

COMP 421: Files & Databases

Fall 2025, Instructor Benjamin Berg, COMP421.001.FA25

Midterm Exam

Name:

PID:

ONYEN:

Question	Points Recvd / Points Possible	Time Estimate
Q1: Mult. Choice	/ 10	10 minutes
Q2: Storage Manager	/ 25	13 minutes
Q3: LSM Trees	/ 25	12 minutes
Q4: B+ Trees	/ 27	23 minutes
Q5: Joins	/ 13	12 minutes

This is your midterm exam. There are many like it, but this one is yours. There are 5 questions, each with several parts/sub-questions. Detailed point values are given for each part in the exam if you are curious. We are providing time estimates for each question based on how long the TAs took when demoing the exam. Of course, you can allocate your time however you want, and different people work at different speeds. The questions are ordered by when the material was discussed in class.

Before you begin, please read and sign:

In accordance with the UNC honor code, I certify that I will not give or receive help on this closed-book exam. No calculators, computers, or mobile devices of any kind are allowed. You may have unlimited blank paper for calculations or extended answers. Mark any extended answers clearly.

Name: _____

The Choice Is Yours (Revisited)

Let's begin with some multiple-choice questions about high-level DBMS design, SQL, and the relational algebra (each question is worth 2 pts).

1. In a disk-oriented DBMS, which of the following statements is FALSE?

Choices:

- A. Random access on disk is much slower than sequential access.
- B. Database page sizes, OS page sizes, and disk page sizes are not guaranteed to have the same size
- C. The buffer pool manages pages fetched from disk into memory.
- D. The OS knows not to double-cache pages held in memory by the buffer pool manager

2. Consider a database with the following relations:

Student:

pid (BIGINT)	onyen (VARCHAR(10))	dept (VARCHAR(10))
--------------	---------------------	--------------------

Enroll:

pid (BIGINT) (FOREIGN KEY REFERENCES (Student(pid)))	course_number (VARCHAR(10))
--	-----------------------------

Consider the relational algebra expression:

$$\pi_{\text{name}}(\sigma_{\text{dept}='CS'}(Student) \bowtie Enroll)$$

This statement returns the result set requested by which of the following SQL queries (circle all that apply):

- A. SELECT dept FROM Student WHERE name='CS';
- B. SELECT name FROM Student AS s INNER JOIN Enroll AS e ON s.pid=e.course_number WHERE s.dept='CS' ;
- C. SELECT name FROM Student AS s NATURAL JOIN Enroll WHERE s.dept='CS';
- D. SELECT name FROM Student AS s, Enroll AS e WHERE s.dept='CS' AND s.pid=e.pid;

3. A hard disk spins at 15,000 RPM, with an average seek time of 2 ms, and a sequential read speed of 300 MB/s. We assume the read head waits for half a rotation on average before the desired sector arrives. Estimate the total time to read a 4 KB block from disk.

Choices:

- A. 0.5 ms
- B. 4 ms
- C. 9 ms
- D. 12 ms

4. Which of the following statements about buffer pool management is true?

Choices:

- A. A dirty page must be written back immediately when modified.
- B. The DBMS can prefetch pages based on the query plan to minimize I/O stalls.
- C. Most production systems rely on the mmap syscall to implement their buffer pool manager
- D. The buffer pool manager handles requests for tuple data, but index data is always managed separately

5. Which of the following statements accurately compares the n-ary storage model (NSM, row store) and the decomposition storage model (DSM, column store). Check all that apply.

Choices:

- A. The NSM is better suited for write-heavy (insert, update, delete) workloads, while the DSM is better suited for read-only analytics workloads.
- B. The NSM commonly uses an unsorted heap file, while the DSM stores data in sorted order
- C. The NSM easily supports relational algebra / SQL queries, while the DSM is used in “no SQL” databases
- D. The NSM tends to allow for better data compression than the DSM

Cache Rules Everything Around Me

For this question, we will consider the heap file storage model discussed in the lecture. Assume we are in a disk-oriented database composed of DRAM and a hard disk drive (HDD). Assume the database uses 4 KB pages, which are accessed via a buffer pool manager with B buffers/frames.

(a) Slotted Page Arithmetic

A 4 KB page uses a slotted-page layout. Tuples are a fixed length of 220 B, including tuple headers, the NULL bitmap, and all tuple metadata.

Given a page with the following specifications:

(i) Page header: 64 B (ii) Slot in the slot array: 4 B (iii) 8 tuples

1. Draw a diagram of the page that clearly labels the three above components. You can label additional components if you need to provide context for items (i), (ii), (iii) (3 pts)

2. How much free space remains in this page? How many additional tuples will fit in this page? (3 pts)

3. Assume that a user deletes the second tuple in the page (as indexed by position in the slot array). Explain one way to perform this delete operation. You can refer back to your drawing in (1) if it helps. There are a few choices of how to implement a delete operation from a slotted page, so describe one pro and one con for the method you chose. (3 pts)

(b) Buffer Pool Manager

The following blocks of code operate on page 0 and page 1, which you can assume have been allocated by the storage manager (e.g., in the `disk_manager` in `bustub`). Assume the buffer pool has a *single* frame/buffer ($B=1$) that begins empty. Consider the following code, which executes sequentially in a single thread:

```
// START T1; Transaction T1
{
    auto writeGuard = bpm->WritePage(0);
    std::string p0 = "page0";
    //copy p0 into page p
    CopyString(p.GetDataMut(), p0)
}
// END T1; writeGuard went out of scope and called unpin on page 0.
//
//=====
//=====
//=====
//
// START T2; Transaction T2
{
    auto readGuard = bpm->ReadPage(1);
    // doing something with this page's data...
    // ...
    //
}
// END T2; writeGuard went out of scope and called unpin on page 0.
```

Consider the state of the buffer pool once execution reaches the comment "END T1;".

1. After T1 ends, what are the pin count and dirty flag values for page 0? (2 pts)

2. If the system crashes between the comments “END T1;” and “START T2;”, is T1’s data (“page0” stored on page_id 0) preserved? (3 pts)

3. If yes, describe exactly when “page0” was written out to disk. If no, describe a solution (e.g., what code or mechanisms you would add to the storage manager) to ensure that this data is written to disk by the end of T1. (3 pts)

(c) Buffer Pool Optimization

1. Recent analytical DBMSs (e.g., DuckDB, or PostgreSQL with “direct I/O”) allow large sequential table scans to bypass the buffer pool and read directly from disk into user memory. Why can bypassing the buffer pool improve performance for workloads that include large scans, and what is one metric you might use to measure this improvement? (4 pts)

2. Most buffer pools use LRU or some LRU approximation to evict pages. Another strategy for mitigating the impact of large scans is to change the eviction policy in some way. How would you change the eviction policy to better handle large scans, and why would you expect this to work? If it helps, you can sketch an example. (4 pts)

A\$AP RocksDB

(a) LSM Design

RocksDB is a popular open source storage system based on the Log-structured Merge Tree (LSM) we discussed in lecture. Let's talk about how this thing works and some optimizations that have been added over the years.

1. The default storage engine for MySQL is InnoDB, which famously uses B+Trees to track the location of each tuple. What are some use-cases/workloads for which you would expect RocksDB to outperform InnoDB, and why? Include which metric you would expect to be better for RocksDB. (4 pts)

2. A potential pitfall of the LSM design is high *Read Amplification*. Explain what read amplification is, how it can occur in an LSM, and one technique used to mitigate it. (4 pts)

(b) Compaction

Because compaction is an important aspect of the LSM design, there have been many compaction algorithms/variants designed to optimize the process.

In lecture, we discussed a simple compaction algorithm. In the literature, the simple algorithm we discussed is known as *tiered compaction*. Tiered compaction works by waiting for the size of a level to exceed some threshold, and then merging all SSTs on that level to form one SST on the next level.

1. When using tiered compaction, how many SSTs need to be searched in the worst case when performing a lookup operation? Select an option and explain your answer. (4 pts)

- a) None
- b) All
- c) One per level
- d) Two per level

RocksDB famously uses *leveled compaction* as its default compaction strategy. Here, when data is flushed from memory, it is inserted at level 0. In level 0, the SSTs can contain overlapping ranges of keys. When the size of any level exceeds its threshold, we pick one file from that level and merge it with one or more files on the *next* level. Merging is performed such that in levels 1 or higher, SSTs within a level contain disjoint (non-overlapping) ranges of keys. No guarantee is made about overlapping ranges *between SSTs in different levels*.

2. RocksDB keeps metadata to track the smallest and largest keys for every SST in the system. In a system with leveled compaction, how would you use this metadata to reduce the number of reads needed during a lookup operation? (5 pts)

4. A downside to leveled compaction is that it is generally shown to have higher write amplification than tiered compaction. Why do you think that is? To show why, it may help to draw a picture of what leveled compaction has to do during compaction to maintain non-overlapping SSTs within each level. (5 pts)

(c) Alternative Storage Models

1. A drawback of RockDB is that, for the most part, it only compresses data at the block level for SST files. Name a storage model that you would expect to be far better at leveraging compression, and explain why/how this model enables compression. (3 pts)

B+Trees Are the Perfect Place for Shade

Assume a disk-oriented DBMS with a Buffer Pool Manager (BPM) and a thread-safe B+Tree that utilizes latch crabbing/coupling, as discussed in lecture. Assume that each node has sibling pointers, meaning this is technically a “B-Link Tree” variant of the original B+Tree.

If it helps, you can use the latching notation:

R(X) acquire read latch; **W(X)** acquire/upgrade write latch; **-X** release

(a) Latch Crabbing

Let’s describe latch crabbing at a high level. For each operation, describe the rules for latching as you traverse the tree from root to leaf. State the rule for when a node is considered “safe” for each operation, and which latches get released when a “safe” node is found. For this question, ignore any splits or merges that may happen once you reach the leaf nodes. (6 pts)

FIND(X)

INSERT(X)

DELETE(X)

(b) B+Tree Design

Consider a B+Tree where each inner node can hold m node pointers. This B+Tree stores n key-value pairs that fit perfectly into N leaf nodes. Assume the inner nodes of the tree are completely full, meaning every inner node has exactly m pointers and $m-1$ keys.

1. Out of the total number of nodes in the B+Tree, what fraction are leaf nodes? You should write your answer in terms of m and N . (6 pts)

Hint: In your answer, you may want to use the geometric partial sum formula:

$$S_L = \sum_{i=0}^L m^i = \frac{m^{L+1} - 1}{m - 1}$$

2. Assume N is large. Does this fraction increase or decrease with m ? Given that a real system might have 8 KB pages holding hundreds of pointers, what does that tell you about the fraction of leaf nodes in a real-world B+Tree? (3 pts)

Zhongrui and Ben are working on a database paper. They want to optimize the *average* index lookup time. Ben proposes using a B+Tree with $m=32$ pointers per inner node to index N leaf nodes. Zhongrui disagrees and makes the following argument: “B+Trees are fine in *worst-case*, but maybe we can improve the *average* lookup time by moving some of the data from leaf nodes to the inner nodes. That way, some FIND operations don’t have to traverse the whole tree”. Zhongrui proposes an alternative index, called a Z+Tree, that uses half of each *inner node* to store key-value data. To make space for the added data in each inner node, the Z+Tree stores only $m=16$ pointers in each inner node instead of 32. The Z+Tree still stores copies of all key-value pairs in the leaf nodes to allow for fast sequential scans, so there are still N leaf nodes. Assume that N is large for the following questions.

3. Using your formula above, how does the fraction of nodes that are leaf nodes compare between the B+Tree and the Z+Tree? (3 pts)

4. How does the height of the B+Tree compare to the height of the Z+Tree? (3 pts)

5. Based on your answers to (3) and (4), do you think that the Z+Tree will improve the average time for a lookup operation? Will Zhongrui’s paper get published? (6 pts)

A Spike Lee Join

For this question, we'll consider doing some joins on the IMDb dataset. We are interested in the following two (simplified) tables.

Directors

director_id (BIGINT)	name (VARCHAR (100))	born (DATE)
----------------------	----------------------	-------------

Films

film_id (BIGINT)	director_id (BIGINT, references Directors (director_id))	title (VARCHAR (200))	released (DATE)
------------------	--	--------------------------	-----------------

There is a B+Tree index on the Directors table, on the director_id key.
There is a B+Tree index on the Films table, on the film_id key.

(a) Join algorithm selection

For each scenario below, state which of the following algorithms you would use, and why, in one or two sentences. It is not necessary to compute explicit run times for this question.

Possible Join Algorithms

- Simple Hash Join
- Partition/GRACE Hash Join
- Sort-Merge Join
- Indexed Nested Loop Join

Scenarios

1. You want to find the names of the 24 feature films directed by Spike Lee. Your SQL query execution plan is first to filter the rows of **Directors** to find Spike's **director_id**, and then join this result set against the **Films** table on the **director_id** column. (2 pts)
2. You want to find all films directed by people born since 1957. You again begin by filtering the **Directors** table and find that there are tens of thousands of these directors. You then join this result set against the **Films** table on the **director_id** column. Assume that each director has directed multiple films during their career. (2 pts)
3. Assume that both **director_id** and **film_id** are assigned chronologically, so that more recently debuting films and directors have higher ID numbers. You want to return the films from the 1000 most recently-debuting directors in chronological order. (2 pts)

(b) I/O cost computations

You write a SQL query to generate the set of all films released since 1989, along with the names of their directors. Your query plan suggests a block nested loop join. After filtering the films table by date, you see the following statistics

Films (filtered by date):

1400 pages, 25000 tuples

Directors (unfiltered):

600 pages, 20000 tuples

1. Assuming you have 102 buffer pages, estimate the number of I/Os for this join. Explain which table you would use as the inner/outer table and why. (3 pts)

2. You believe a hash join of some kind would work better here. Describe the hash join algorithm you would use here and why. Specify which table is used as the inner/outer table in your solution. Estimate the number of I/Os your solution requires. You can assume perfectly uniform (unskewed) hashing. (4 pts)

</END OF EXAM>